

Third Milestone (Team 1)

Adrian Tirpak

Johannes Hammermann

◆ Current Progress Overview

Milestone 2

01.06.26



Milestone 3

22.06.26



Final Deadline

14.07.26

■ Set Up Architecture

- Including UI frontend, backend, Pi server

■ Tested Manual Keyboard Control

- Controlled robot using keyboard without UI, safety limits through calibration

■ Running Octo Zero-Shot

- Explored running Octo without finetuning



■ Architecture Improvements

- Added UI features, webcams, workstation client

■ Precise Movement Control

- Controlled robot using controller and UI, absolute safety limits using encoder

■ Fine-Tuning Octo

- Ran first data collection trial, fine-tuned Octo on it

UI Changes

The screenshot displays the Robot-Team1 control interface. The top bar shows 'Robot-Team1' and 'DEVICES ONLINE: 2' with a 'CONNECTED' status. The main interface is divided into several sections:

- TELEMETRY:** Displays joint angles (JOINT (C), JOINT (B), JOINT (A)) and positions (X, Y, Z) in degrees and meters.
- MASTER CONTROL:** Includes buttons for 'KEYBOARD', 'CONTROLLER', 'HOME', 'SET ZERO', 'START CAPTURE', and a prominent red 'EMERGENCY STOP' button.
- MANUAL JOG:** Features a PWM slider set to 45% and buttons for joint selection (C SHOULDER, B ELBOW, A WRIST) and movement (A, D, W, S, E, Q).
- CAMERA FEED:** Shows a video stream area with an 'ENABLE' button and a 'STREAM PAUSED' message.
- KINEMATIC PROFILE:** Includes a '3-DOF Schematic' and buttons for 'SIDE' and 'TOP' views.
- TERMINAL:** A console window at the bottom showing the following log:

```
> stop 7:58:47 PM
# [CONSOLE] Emergency stop command received. 7:58:47 PM
# [CONSOLE] Mock: Emergency stop active. All PWM is blocked. Further stop presses do nothing. Type 'unblock' in the UI console to reactivate the robot. 7:58:47 PM
> unblock 7:58:51 PM
# [CONSOLE] Unblock command received. 7:58:51 PM
# [CONSOLE] Emergency stop cleared. Robot PWM is reactivated. 7:58:51 PM
> [Local Client] Camera streams: OFF 7:59:01 PM
> type command
```

Fig. 1: Updated UI with PWM scaling, controller support and kinematic preview

◆ Safety Limits

- We need to know robot position to enforce hardware limits
 - Motors are equipped with absolute rotary encoders
 - Problem: when using 5:1 gear ratios, a 100° robot angle change means 500° motor movement
- We need to count revolutions

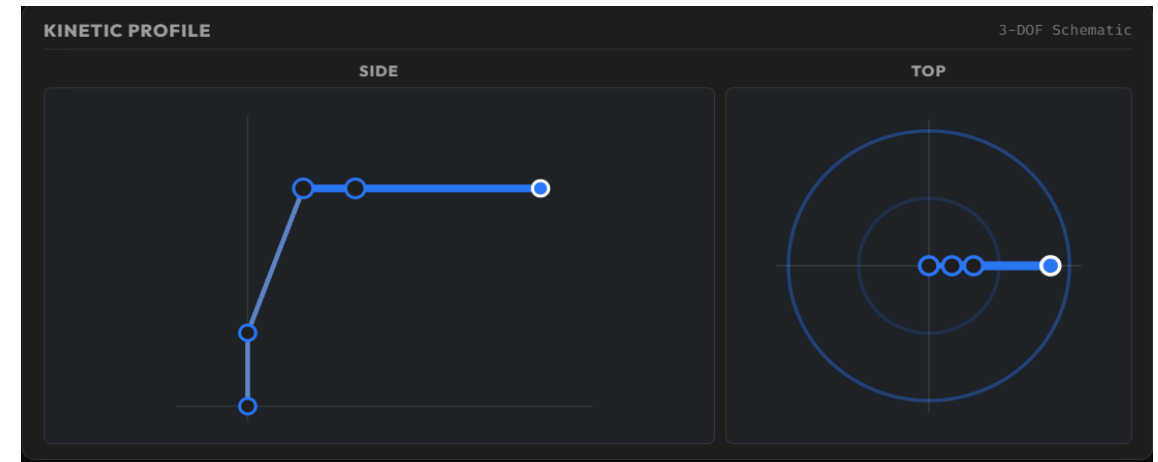


Fig. 2: UI preview of current robot position

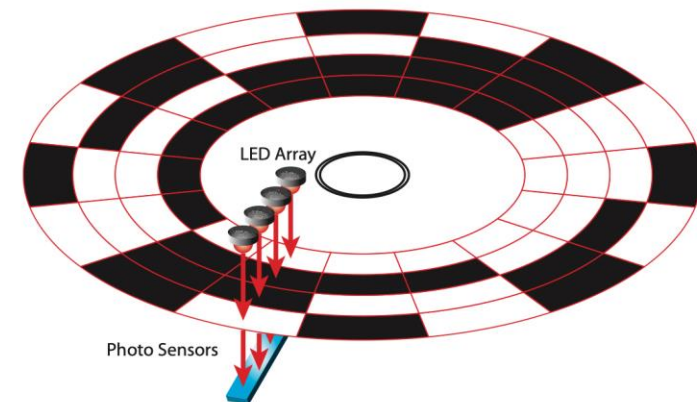


Fig. 3: Schematic of absolute rotary encoder [1]

Witness Marks

- BuildHat has a built-in function for counting revolutions, but 'zero' resets on every Pi reboot
→ Simple first solution: use witness marks to define the zero-position

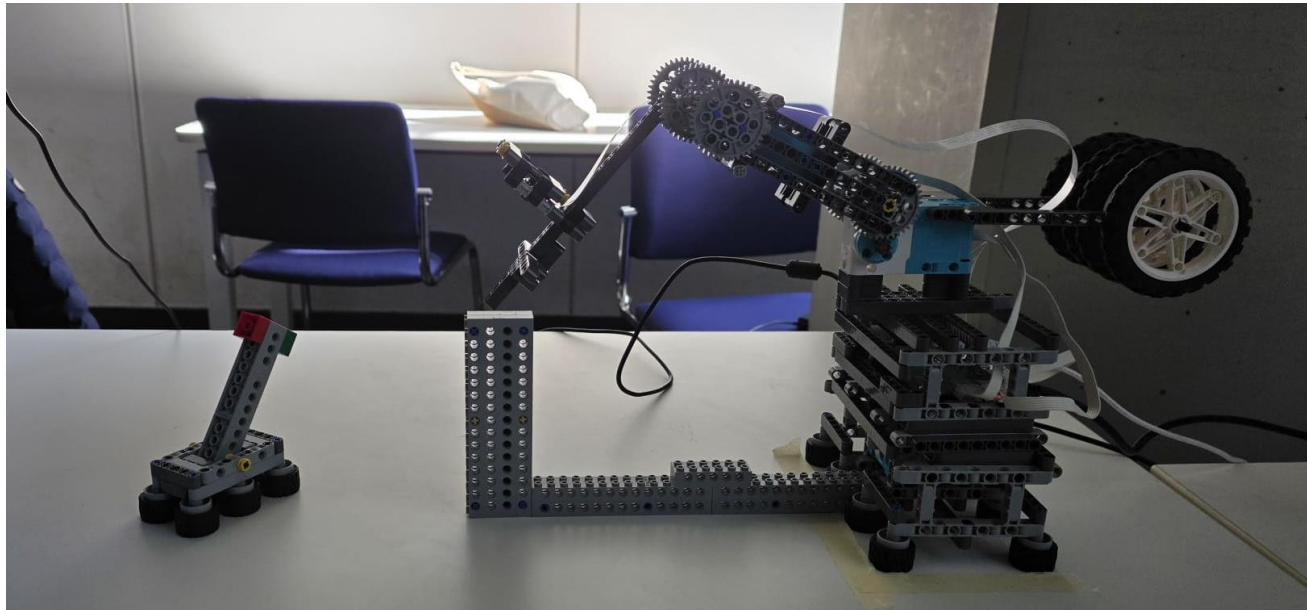


Fig. 4: End effector witness mark

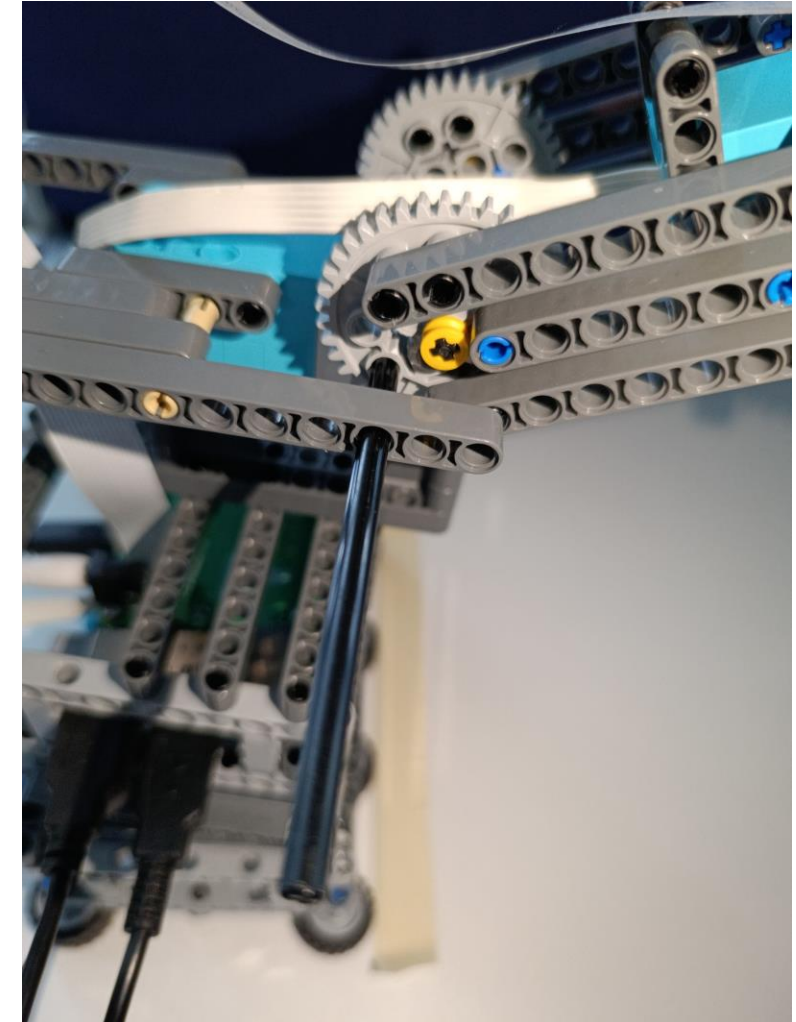


Fig. 5: Elbow witness mark

Absolute Limits

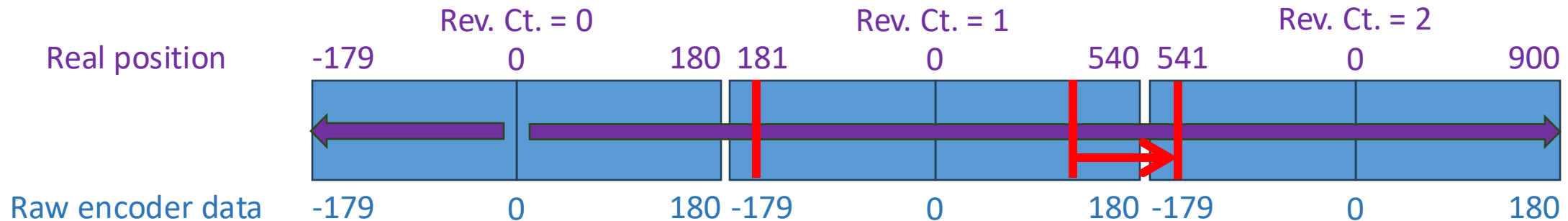
- Problem with Witness Marks: they need recalibration on every server restart
 - We count revolution counts (Rev. Ct.) ourselves and save them as a file when shutting down

◆ Absolute Limits

- Problem with Witness Marks: they need recalibration on every server restart

→ We count revolution counts (Rev. Ct.) ourselves and save them as a file when shutting down

In case the robot gets moved slightly while turned off: How to prevent errors?



The Rev. Ct. was saved as an example: Robot moved from 180° to -179° but the encoder outputs a 330° to 550° while off new position! Are we here?

Fig. 6: Calculating pos. from raw data

◆ Absolute Limits

- We not only save the Rev. Ct., but also the raw encoder data when shutting off
 - When turning back on, we warn in the UI if the position differs by more than 20°
- Small differences are accounted for by correcting the revolution count

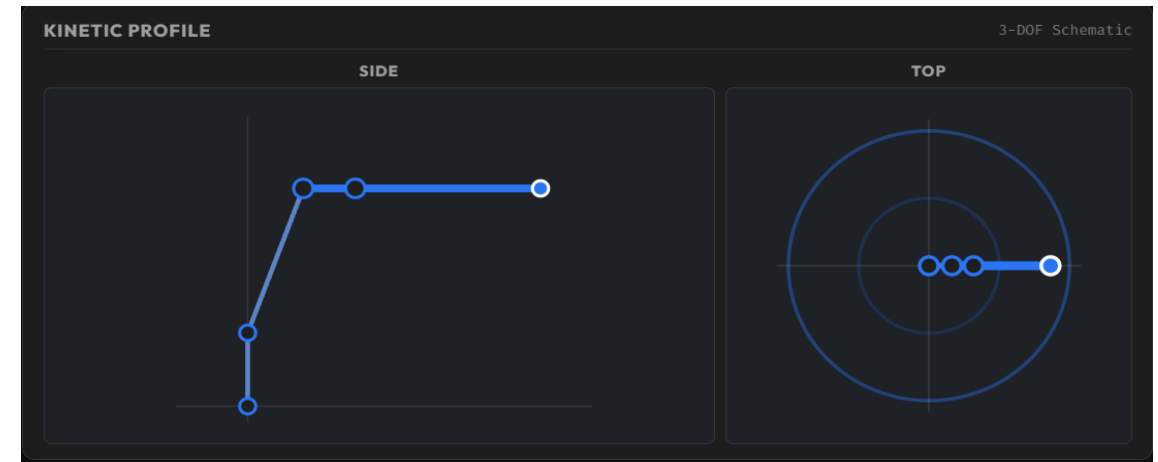
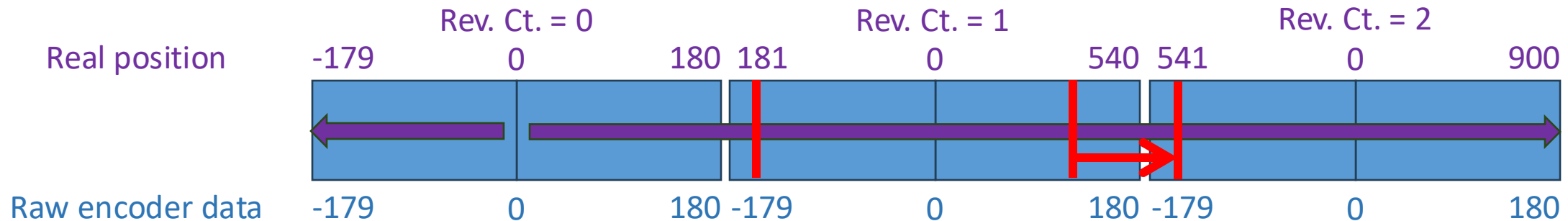


Fig. 2: UI preview of current robot position



The Rev. Ct. was saved as 1,
but the encoder outputs a
new position! Are we here?

Fig. 6: Calculating pos. from raw data

◆ Teleoperation Rework

First Iteration

- Calculation of motor target speeds/angles
 - Use of motor's built-in PID controller with `motor.run_for_degrees()`
- Imprecise movement, unreliable when changing direction

New Iteration

- Low-level motor control using PWM drive values
 - Bypasses PID controller with raw motor commands
- Responsive and smooth movement even when changing direction

◆ Teleoperation Infrastructure

Client-Server Architecture

- GUI + controller script running on client (e.g. laptop)
- Web server running on Pi receives remote commands
- Motor commands rerouted to Raspberry Pi Build HAT

Controller Input Processing

- Normalized joystick inputs processed with deadzones & input smoothing
- Translated into individual motor jog speeds sent over WebSocket
- Additional functionality for safety and recording commands

Intuitive control on all 3 axes at once

◆ Teleoperation Controls

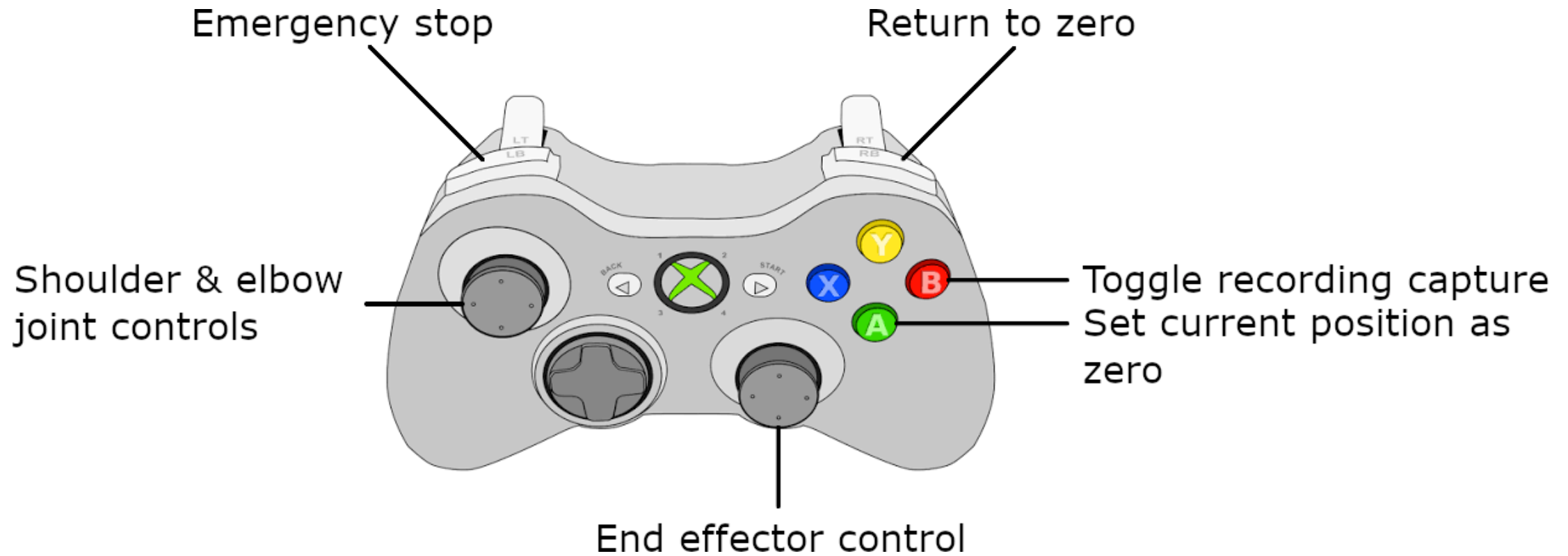


Fig. 7: Control Scheme [2]

◆ Training Octo

- Octo pre-trained on different robot embodiments and tasks
 - Goal: fine-tune Octo to fit our robot and task
 - Manual demonstrations required to use as training data
- Octo learns to control robot arm using inference

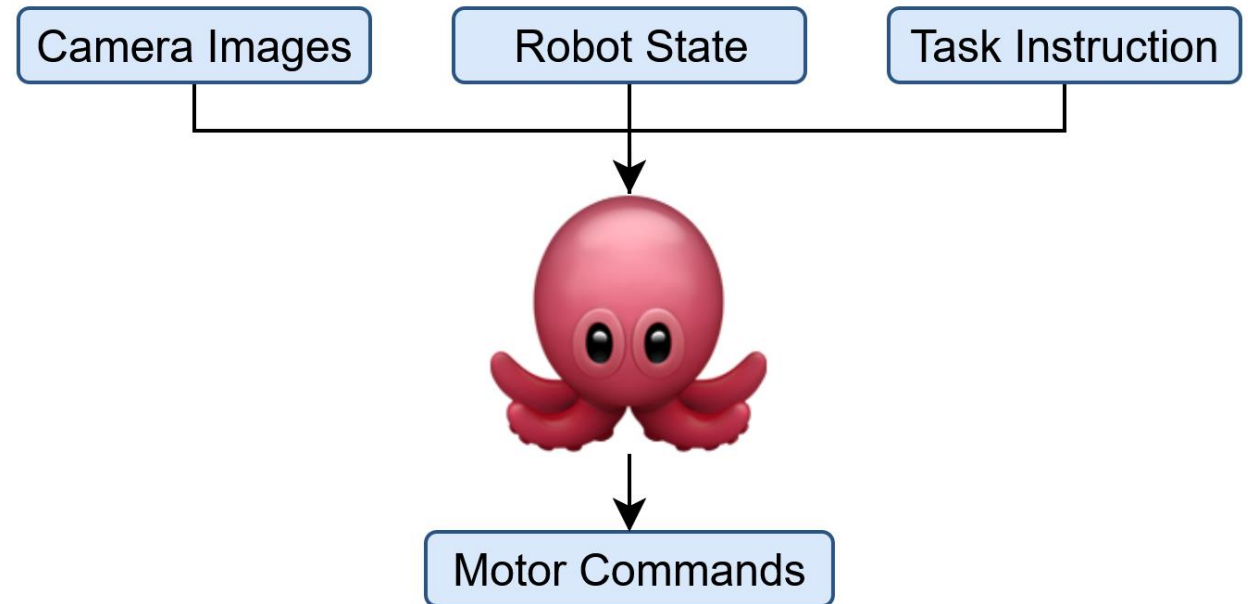


Fig. 8: Octo [3]

◆ Data Recording

- First test recording session with 32 episodes
- Visual input via stable overhead camera and wrist camera
- Variations:
 - Prompts
 - Arm starting positions
 - Lever positions and rotations
 - Joint movements
 - Distractor objects
- Brightened & darkened images for training

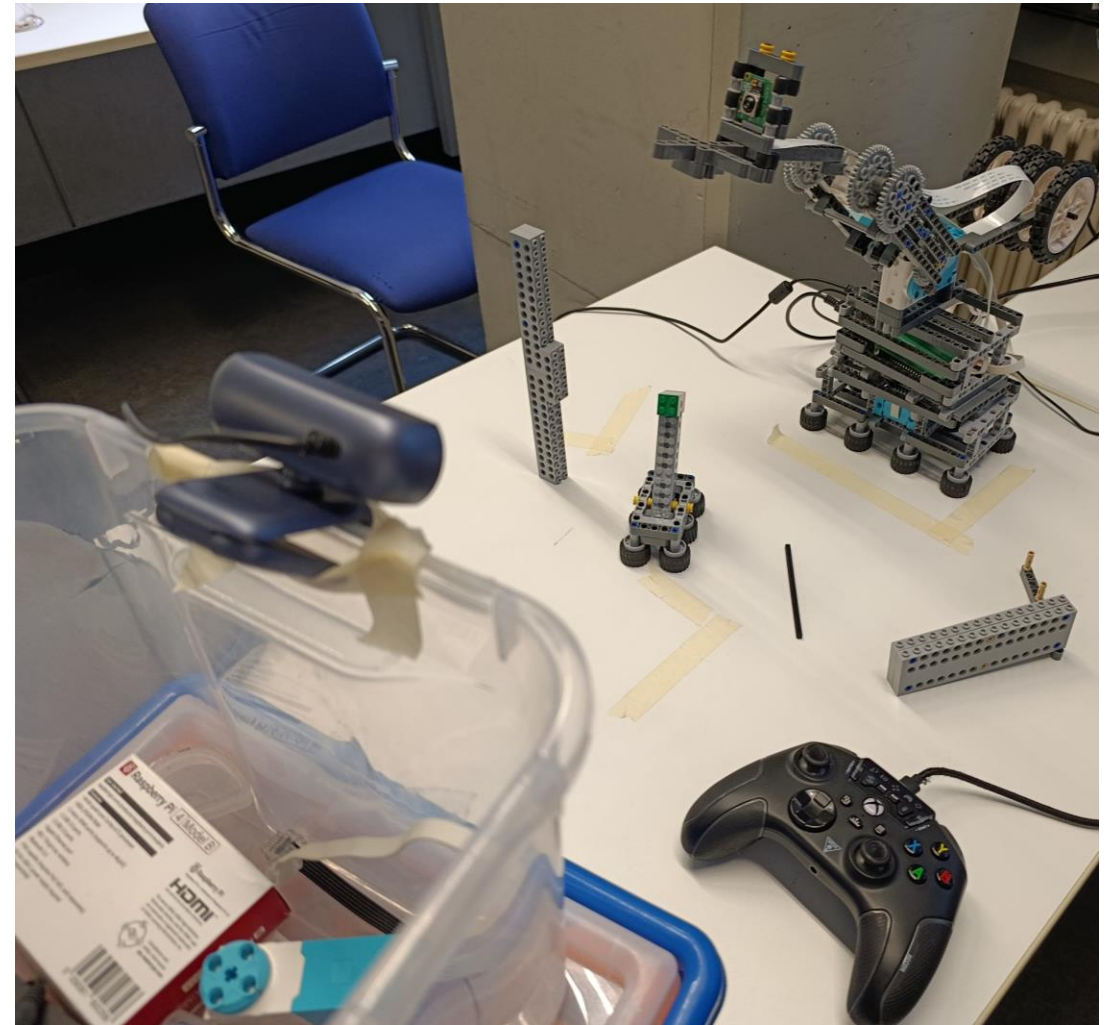


Fig. 9: Data Recording Setup

◆ Fine-tuning

- Both model sizes (Octo Small & Base)
- Hyperparameter sweep with 26 different configurations, e.g.:
 - Batch size
 - Warmup steps
 - Weight decay
- Baseline: Octo Small with default settings
- Evaluation of training & validation loss
- Analysis of randomness impact by varying baseline seeds

```
100%|██████████| 5000/5000 [11:10<00:00, 7.46it/s]
I0619 13:46:21.382212 132045361911616 finetune.py:286] Finetuning complete. Saved checkpoints to /save
wandb:
wandb: 🚀 View run finetune_switcher at:
wandb: Find logs at: wandb/run-20260619_133433-igsupu08/logs
(base) lego-team-1@themachine:~/robot/src/octolego-ees/finetuning$ |
[finetune]0:bash*
```

Fig. 10: First fine-tuning run

◆ Fine-tuning Evaluation

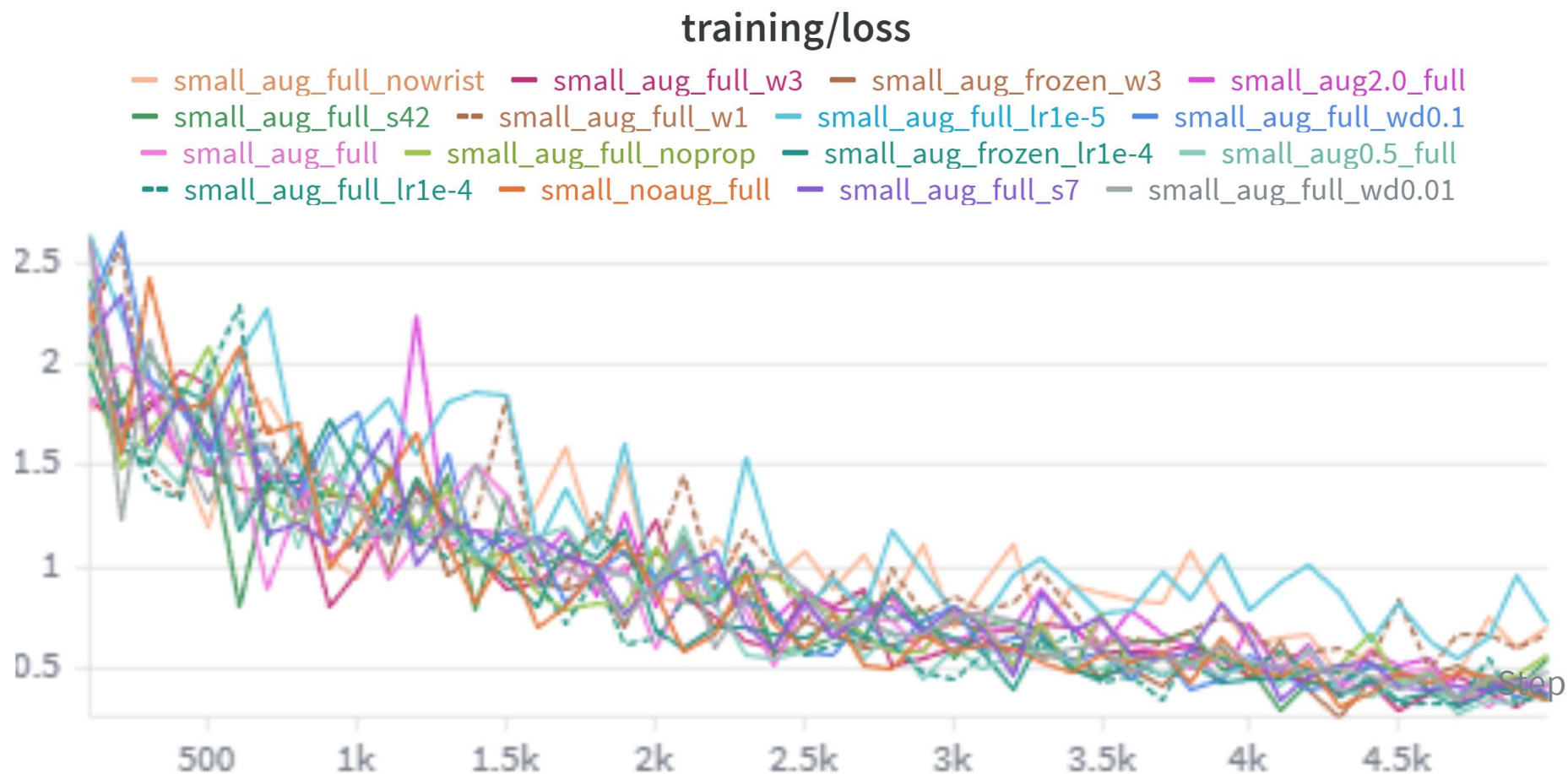


Fig. 11: Training loss graph

◆ Fine-tuning Evaluation

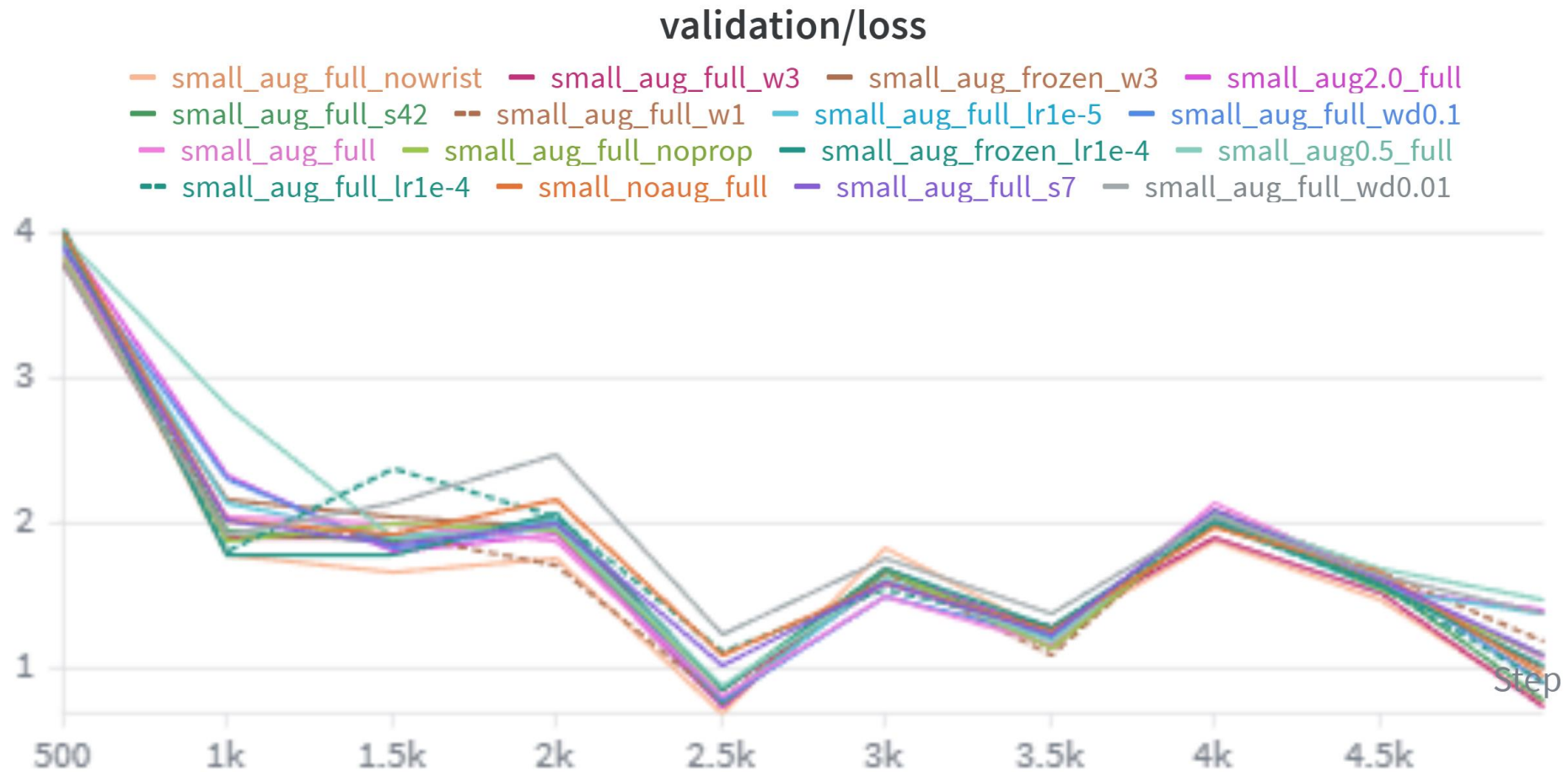


Fig. 12: Validation loss graph

◆ Fine-tuning Evaluation

- Best results using 500 warmup steps instead of 100
 - Validation loss improvement of 0.17 relative to baseline
 - Changing training seed on baseline
 - Validation loss variation of 0.27
- Improvement may be random noise!

◆ Outlook

Milestone 2

01.06.26



Milestone 3

22.06.26



Final Deadline

14.07.26

■ Architecture Improvements

- Added UI features, webcams, workstation client

■ Precise Movement Control

- Controlled robot using controller and UI, absolute safety limits using encoder

■ Fine-Tuning Octo

- Ran first data collection trial, fine-tuned Octo on it



■ Recording Trials

- Using the UI with a controller to record trials

■ Movement Through Octo

- Building a timed queue for moving towards Octo inference outputs

■ Extensive Octo Fine-Tuning & Evaluation

- Training and evaluating Octo on the whole dataset

◆ Reference List

- [1] Rotary Encoder: <https://www.linearmotiontips.com/wp-content/uploads/2021/01/US-Digital-Absolute-Encoder-Disk.jpg> (Accessed: 20.06.2026)
- [2] Control Scheme: https://commons.wikimedia.org/wiki/File:Xbox_Controller.svg (Accessed: 20.06.2026)
- [3] Octo: <https://octo-models.github.io/> (Accessed: 21.06.2026)
- Octo (Paper): <https://arxiv.org/pdf/2405.12213> (Accessed: 21.06.2026)

Questions?



◆ Manual Control Demonstration

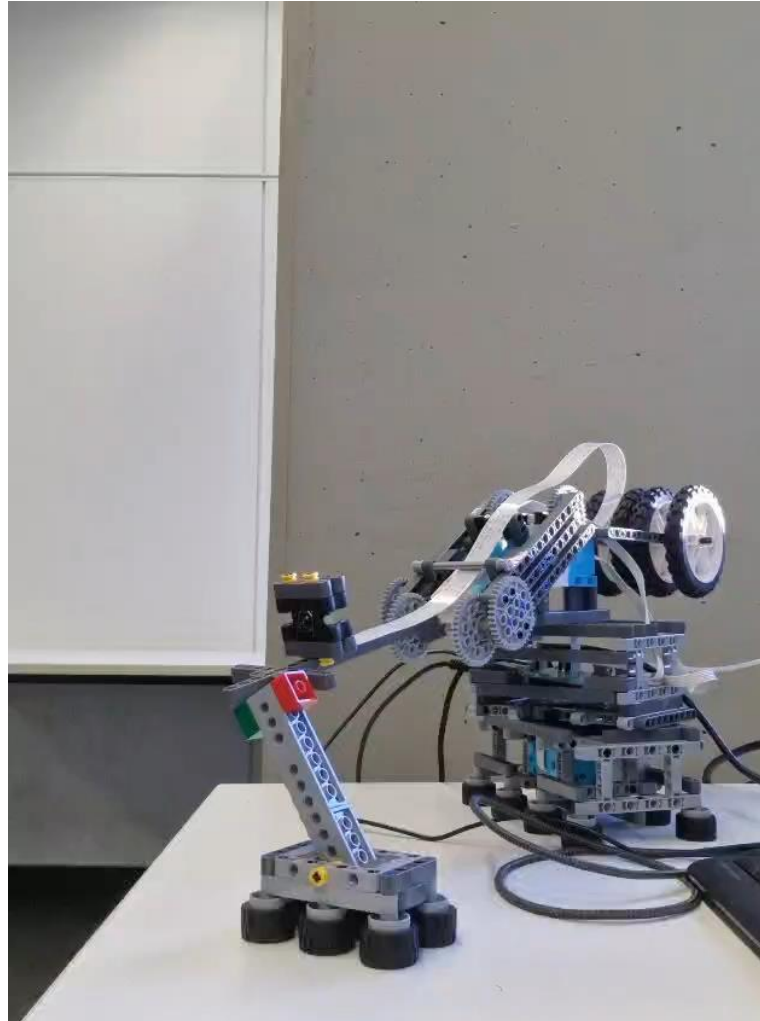


Fig. 13: Manual Control Demonstration